

SE Web Toolkit

내부 엔지니어링 도구를 위한 디자인 시스템과 Claude Code 플러그인. 목표는 개발 속도, **양산형이 아닌 서비스별 개성을 가진 SE 아이덴티티**, 전문가 수준의 UI 품질, 그리고 낮은 진입 장벽이다.

FE 표준 React + Vite + TypeScript 기존 서비스 React 계열 백엔드·배포 비표준 (무관해야 함)

배포 Claude Code 플러그인 (marketplace)

한 줄 요약

"같은 가족, 다른 형제." SE 공통 골격(타이포·간격·컴포넌트 형태·AppShell)은 패키지로 고정하고, 서비스마다 아이덴티티(액센트·마크·시그니처 요소·톤)를 *반드시* 다르게 정하게 한다. Claude는 웹 디자이너처럼 **디자인 플랜** → 구현 → **스크린샷 비평** → 수정 루프를 돌고, 규칙은 린터와 훅이 기계적으로 지킨다.

1

아이덴티티를 2층으로 나눈다

브랜드 코어(고정) / 서비스 아이덴티티 (가변, 필수). 개성은 옵션이 아니라 필수 입력이다.

2

디자인 품질은 루프로 만든다

코드 생성 한 번으로 전문가 결과가 나오지 않는다. 플랜 → 스크린샷 → 비평 → 수정.

3

예제가 품질의 상한이다

Claude는 예제를 복사·변형한다. 레퍼런스 앱과 패턴 예제를 최고 수준으로 먼저 만든다.

4

규칙은 린터가 지킨다

LLM은 문서를 잊지만 exit code는 못 잊는다.

```
@se/eslint-plugin
```

+ PostToolUse 훅.

문제 → 설계 원칙

반복되는 문제	설계 원칙	구현 수단
초기 설정을 매번 새로 작성	0분 셋업	<code>create-se-app</code> 템플릿 + <code>/se:new</code>
서비스마다 UI 스타일이 다름	브랜드 코어 고정	<code>@se/tokens</code> , <code>@se/ui</code> , ESLint 강제

■ 브랜드 코어 — 모든 서비스 동일 (패키지가 강제)

- **타이포 체계** 본문 Pretendard + IBM Plex Mono, 스케일 12/13/14/16/20/24/32, 행간·자간
- **공간 체계** 4pt 리듬, 간격 스케일, 반경 4/6/10, 보더·그림자 단계
- **컴포넌트 해부학** 버튼·입력·표·다이얼로그의 형태, 크기, 상태, 동작
- **AppShell 골격** 좌측 네비 + 상단 검색(🔍), 사용자 메뉴, "마크 + 서비스명" 로크업 위치
- **의미 색**
success/warn/danger/info, 작업 상태색 — 액센트와 분리
- **인터랙션·카피 관례** 단축키, 위험 동작 확인, 토스트, 3상태 구조, 모션, 한국어 문체

■ 서비스 아이덴티티 — 서비스마다 다르게, 반드시 정함

- **액센트 hue** 채도·명도 밴드 고정, hue 자유. 레지스트리의 형제와 30° 이상, **의미 색**
(danger ≈ 30° · warning ≈ 70° · success ≈ 155° · info ≈ 260°)
과 18° 이상 거리 — 가까우면 액센트 칩이 경고·오류처럼 읽힌다 (호박 45°에서 실제로 겪음)
- **뉴트럴 편향** 액센트 방향으로 2~3% 기울인 회색
- **마크** 모노그램 또는 아이콘 + 이름 로크업
- **시그니처 요소 1개** 상태 스트립 · 검색 히어로 · 단계 레일 · 타임라인 리본 · 지표 마키 중 택1
- **기본 밀도** compact / comfortable
- **헤딩 폰트** 한글 호환 승인 목록 3~4종 중 택1 (본문은 고정)
- **차트 팔레트** 액센트 순차 / 카테고리 변형
- **톤** 마이크로카피·빈 상태·일러스트의 목소리

개성이 나오는 원리. 슬롯이 8개고 각 슬롯의 선택지가 3~8개면 조합은 수천 가지지만 골격은 하나다. 옆에 두면 "같은 팀 제품"이고, 따로 보면 "다른 제품"이다.

구현

```
// se.identity.json (프로젝트 루트)
{
  "id": "job-monitor",
  "name": "Job Monitor",
  "mark": { "type": "monogram", "text": "JM" },
  "accent": { "hue": 195 }, // 채도·명도 고정, hue 자유
  "neutralBias": "cool",
  "signature": "status-strip",
  "density": "comfortable",
  "displayFont": "pretendard",
  "chart": "accent-sequential",
  "tone": "terse"
}
```

- @se/tokens 의 createTheme(identity) 가 CSS 변수 세트를 생성. WCAG AA 대비와 의미 색 hue 거리는 빌드 시 검증되어 실패하면 빌드 실패.
- 툴킷 저장소의 identities/registry.json 에 서비스별 아이덴티티 등록. /se:identity 가 hue 충돌·중복 시그니처를 검사.
- 시그니처는 @se/ui/signatures 의 검증된 컴포넌트에서 고른다. 자유 발명이 아니라 승격 절차로 목록을 키운다.
- 각 프로젝트는 개발 전용 /__identity 라우트에서 자기 아이덴티티 시트(팔레트·마크·타입·시그니처·상태색)를 본다.

디자인 품질 — Claude가 웹 디자이너처럼 일하게 하는 장치 핵심

"전문가가 만든 것 같다"는 것은 결과물의 속성이 아니라 **작업 순서**의 속성이다. 디자이너가 하는 일을 순서대로 스킬에 넣고, 마지막에 눈으로 확인하는 단계를 자동화한다.

se-design 스킬 — 자동 트리거, se-ui 와 짝

① 코드 전에 디자인 플랜 (필수, 10줄 이내)

- 이 화면의 **단 하나의 목적**, 첫 3초에 봐야 할 것
- 주 액션 1개, 보조 액션, 위험 액션
- 정보 계층 3단계(요약 → 목록 → 상세), 레이아웃 개념 한 문장
- 밀도, 액센트를 쓸 **한 곳**
- 아이덴티티 시그니처를 이 화면에서 어떻게 드러낼지

② 구성 규칙 — 디자이너의 기본기

- 4pt 리듬, 타입 스케일 이탈 금지, 레이아웃은 grid/flex + gap
- 한 화면에 액센트는 한 곳. 의미 색은 액센트가 아니다
- 모든 블록을 카드로 만들지 않는다. 보더·배경·그림자는 역할별로 하나
- 숫자는 `tabular-nums`, ID·해시·경로는 모노스페이스
- 표는 조용하게: 바깥 테두리 없이 행 구분선만, 텍스트 좌·숫자 우, 상태는 pill
- 요약은 상세보다 위에. KPI 타일은 숫자가 목적일 때만
- 빈 상태엔 **다음 행동**, 로딩은 형태를 유지한 스켈레톤, 에러는 원인 + 해결
- 모션은 상태 전이에만 150–200ms. 카피는 디자인 재료

④ 내부 도구 정보 설계

- 읽히는 화면이 아니라 **스캔되**는 화면
- 상태를 형태로: pill · 스트라이프 · 점
- 1차 분류는 탭, 2차 필터는 표 바로 위 한 줄, 상세는 드로어/분할
- 키보드로 전부 가능, 시간은 상대 + 절대 병기
- 위험 동작은 이름을 다시 입력

③ 안티패턴 — "AI가 만든 티"를 금지

~~이미지 아이콘~~

~~전부 가운데 정렬~~

~~그라디언트 히어로~~

~~모든 블록이 같은 등근 카드~~

~~보라·파랑 조합~~

~~데이터 없는 KPI 타일 나열~~

~~설명 없는 대시보드~~

~~100% 화색 뉴트럴~~

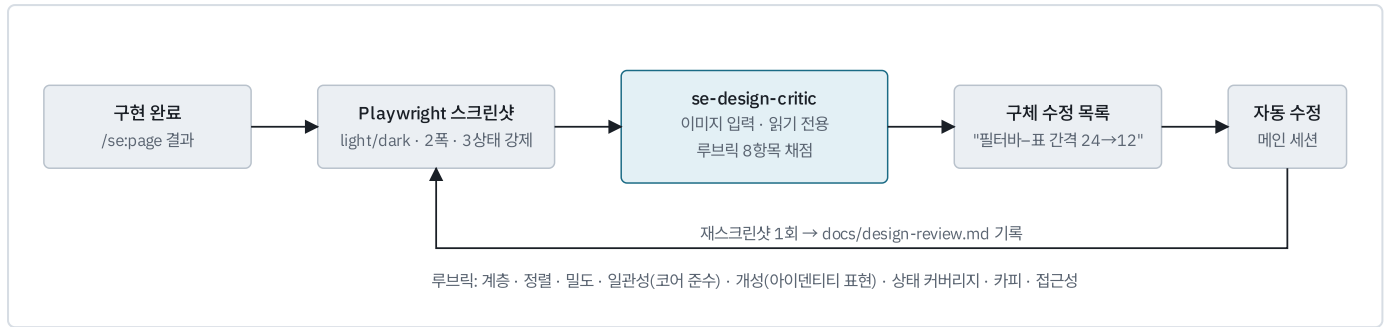
~~내용 없는 "환영합니다" 화면~~

~~섹션마다 01 02 03 장식~~

~~모든 것에 그림자~~

~~표만 화면 끝까지 짝 찬 목록~~

시각 리뷰 루프 — /se:review

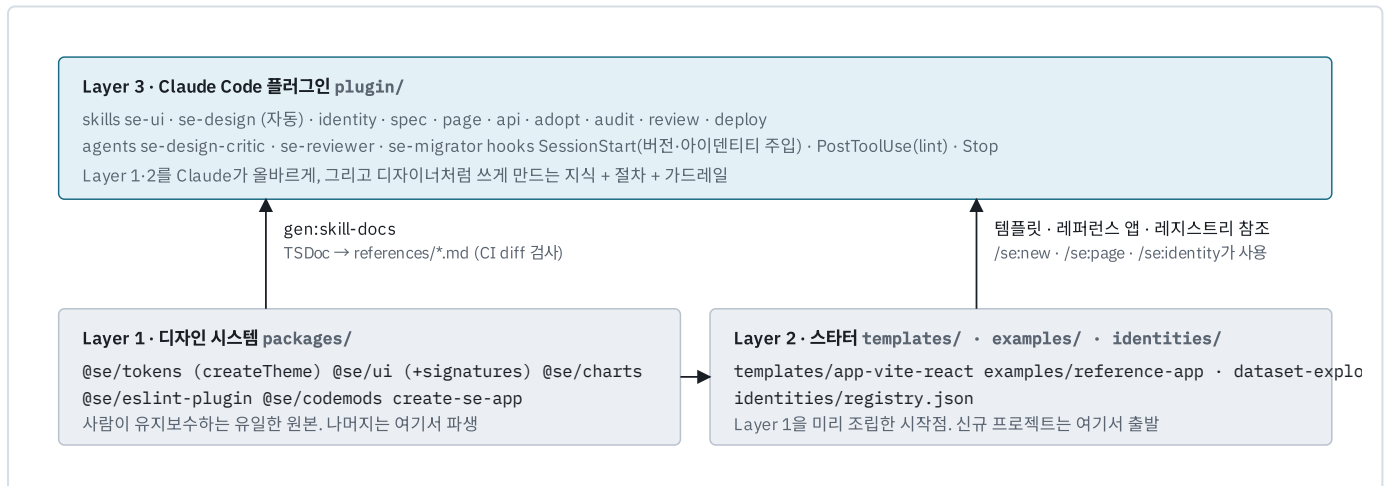


사람이 하던 디자인 QA를 매번 자동으로 돌린다. 비평은 읽기 전용 에이전트가, 수정은 메인 세션이 맡아 책임을 나눈다. 루브릭 점수는 /se:audit 에 합산된다.

레퍼런스 앱 — 품질의 상한선

examples/reference-app 은 여섯 패턴 전부를 최고 수준으로 구현한 "Job Monitor" 완성본이다. Storybook·스킬 예제·템플릿의 공통 소스이며, **Claude가 만드는 모든 화면은 이 앱의 변형이다**. 따라서 P1에서 가장 많은 디자인 공수를 여기에 쓴다 — 필요하면 디자이너가 1회 참여한다. 두 번째 형제 examples/dataset-explorer 는 "같은 가족, 다른 형제"의 증명이자 검색 히어로·DetailPage 패턴의 원본이다.

전체 구조 — 3개 레이어



Layer 1은 사람이 유지보수하는 유일한 원본이다. 플러그인의 컴포넌트 지식은 코드에서 생성되므로 스킴이 코드와 어긋날 수 없다.

Layer 1 — 디자인 시스템 packages/

@se/tokens — 4계층

primitive → **brand** (고정) → **identity** (서비스별, createTheme이 생성) → semantic (bg.surface, fg.muted, accent.default, status.*) → component . W3C DTCG JSON → Style Dictionary → CSS

변수 + Tailwind v4 preset + TS 상수. 코드에서는 semantic만 사용하며(린트), `[data-theme]` · `[data-density]` 속성으로 전환한다.

@se/ui — 컴포넌트 3티어 + 시그니처

API는 shadcn/Radix 관례를 그대로 따른다. Claude가 이미 아는 이름이라 생성 정확도가 높고 학습 비용도 낮다.

티어	목록
T0 프리미티브	Button, Input, Textarea, Select, Combobox, Checkbox, Switch, RadioGroup, Badge, Tooltip, Popover, Dialog, Sheet, Tabs, Table, Card, Toast, DropdownMenu, Command, Skeleton, Alert, Separator, Avatar
T1 내부도구 복합	DataTable (TanStack: 서버 페이지네이션·정렬·컬럼 토글·행 선택·CSV), FilterBar, StatusBadge, DescriptionList, JsonViewer, LogViewer(가상 스크롤·ANSI), StatCard, Sparkline, TimeRangePicker, CodeBlock, EmptyState, ErrorState, ConfirmDialog, Form(react-hook-form + zod)
T2 레이아웃 / 패턴	AppShell, ListDetailPage, DashboardPage, DetailPage, FormWizardPage, SettingsPage — 각 패턴은 <code>examples/</code> 에 로딩·빈·에러 상태를 포함한 완전 예제를 가진다
<code>signatures/</code>	StatusStrip, SearchHero, StageRail, TimelineRibbon, MetricMarquee — 아이덴티티 시그니처 후보

@se/eslint-plugin

규칙	내용
<code>se/no-raw-color</code>	hex/rgb 리터럴, Tailwind 임의값 색상 금지 → 토큰
<code>se/no-raw-control</code>	<code><button></code> <code><input></code> <code><select></code> <code><table></code> 직접 사용 금지 → <code>@se/ui</code>
<code>se/no-primitive-token</code>	primitive 토큰 직접 참조 금지
<code>se/page-states</code>	페이지 컴포넌트에 loading / empty / error 분기 누락 경고
<code>se/import-from-ui</code>	<code>@radix-ui/*</code> 직접 import 금지
<code>se/single-accent</code>	한 페이지 컴포넌트에 primary 버튼 2개 이상 경고

`@se/charts` 는 Recharts 래퍼다. 색은 전부 CSS 변수(의미 색은 상태에만, 카테고리는 chart-1..8 고정 순서), 축선 없음·가로 점선 격자·얇은 마크, 2개 이상 시리즈는 범례 필수. `ChartCard` (제목·설명·범례·링크), `BarChart` (스택), `LineChart`, `MeterList` (순위·비율). `@se/codemods` 는 jscodeshift 변환 (MUI/antd/styled hex → se)으로 80%를 자동 처리하고 나머지는 `se-migrator` 가 맡는다.

Layer 2 — 스타터 `templates/` `examples/`

`npx create-se-app <name>` 또는 `/se:new` 로 생성한다.

- Vite + React 19 + TS strict, React Router, TanStack Query, react-hook-form + zod

- AppShell이 적용된 `src/app/`, `se.identity.json` (생성 시 `/se:identity` 가 채움), `/__identity` 시트
- **백엔드 무관**: OpenAPI → 타입 클라이언트(`openapi-typescript`), MSW 목 기본 ON. 백엔드가 없어도 UI를 끝까지 만들고 `.env` 의 `VITE_API_BASE` 만 바꿔 연결
- ESLint(`@se/eslint-plugin`) + Prettier + Vitest + Playwright(스크린샷 리뷰 설정 포함)
- Dockerfile(multi-stage nginx) + `nginx.conf` (SPA fallback, `/api` 프록시) + CI 템플릿
- `.claude/settings.json` + `CLAUDE.md` + `docs/spec.md` 템플릿
- **Node 정책 파일 동봉**: `.nvmrc` (22), `engines.node: "22.x"` + `packageManager`, `.npmrc` 의 `engine-strict=true` — 생성 즉시 툴킷과 같은 런타임

Layer 3 — Claude Code 플러그인 `plugin/`

구현 상태 (v0.5): 아래 구조가 전부 존재한다 — 스킬 11개(자동 2 + 명령 9), 에이전트 3, 혹 2, `gen:skill-docs` (43 파일 · 98 컴포넌트, CI `--check`), `create-se-app` (E2E 검증). 템플릿은 플레이스홀더 없는 진짜 워크스페이스 앱이라 루트 `typecheck.lint`가 항상 분다. 미완: `@se/*` npm 배포(그 전까지 `/se:new` 는 모노레포 `examples/` 에 생성), 시그니처 2종, `@se/codemods`.

```

plugin/
├── .claude-plugin/plugin.json  # name
├── skills/
│   ├── se-ui/                [자동] 컴포넌트·토큰·파일
│   │   └── references/      ← gen:skill-docs
│   ├── se-design/           [자동] 디자인 플랜·구성
│   ├── se-identity/         /se:identity
│   ├── se-new/              /se:new
│   ├── se-spec/             /se:spec
│   ├── se-page/             /se:page
│   ├── se-api/              /se:api
│   ├── se-adopt/            /se:adopt
│   ├── se-audit/            /se:audit
│   ├── se-review/           /se:review
│   └── se-deploy/           /se:deploy
├── agents/
│   ├── se-design-critic.md   # 스크린샷 읽기
│   ├── se-reviewer.md       # 코드 규칙·검증
│   └── se-migrator.md        # 파일 단위 마이그레이션
├── hooks/hooks.json
└── README.md
  
```

혹

- **SessionStart** `node -v` 검사(정책 미달이면 경고 + `.nvmrc` 안내) → `@se/ui` 버전 + `se.identity.json` 요약을 컨텍스트로 주입. 플러그인 버전과 불일치 시 경고
- **PostToolUse** (Edit·Write, `*.tsx`) `eslint --fix` 후 잔여 에러를 `exit 2`로 반환 → Claude가 즉시 수정
- **Stop** 변경된 페이지에 상태 누락·액센트 중복이 있으면 마무리 전 경고

`/se:identity` 흐름

1. 인터뷰 5문항 — 서비스가 하는 일 / 주 사용자와 사용 순간 / 분위기 3단어 / 닻으면 안 되는 형제 / 가장 중요한 화면
2. `identities/registry.json` 조회 → 가용 hue 밴드(형제 30°·의미 색 18° 제외)·미사용 시그니처 계산
3. 2-3안 생성 — 액센트·마크·시그니처·밀도·톤이 서로 다르고, 가장 중요한 화면을 각 안으로 렌더한 미리보기
4. 선택 → `se.identity.json` 작성 → 레지스트리 PR

se-ui SKILL.md 골격

```
---
name: se-ui
description: SE 디자인 시스템(@se/ui, @se/tokens)을 쓰는 React 프로젝트에서 UI를 만들거나 수정할
---
## 규칙 (위반 시 린트 실패)
- 색/간격/반경은 semantic 토큰만. hex 금지. 서비스 색은 se.identity.json이 정한다.
- 폼 컨트롤·표·다이얼로그는 @se/ui. raw HTML 컨트롤 금지.
- 모든 페이지는 loading / empty / error 상태를 가진다.
- 새 컴포넌트를 만들기 전에 references/components에 있는지 확인. 시그니처는 signatures/에서만.

## 패턴 선택표
| 목록+필터+상세 → ListDetailPage | 지표·상태 요약 → DashboardPage | 단일 객체+탭 → DetailPage
| 다단계 입력 → FormWizardPage | 설정 → SettingsPage |

## 절차
1. se-design의 디자인 플랜을 먼저 쓴다 2. 패턴 예제를 복사한다 3. 변형한다
4. 3상태를 채운다 5. 스크린샷 1회로 확인한다
```

문서 자동 동기화. packages/ui/src/**/*.tsx (TSDoc + stories) → pnpm gen:skill-docs →
plugin/skills/se-ui/references/** → CI에서 diff 검사. 스킬 버전은 @se/ui 의 major.minor와 같고,
SessionStart 혹은 불일치를 경고한다.

시나리오 A – 신규 프로젝트

/se:new job-monitor	템플릿 생성 → /se:identity 자동 진입(인터뷰 · 3안 미리보기 · 선택) → 첫 실행
/se:spec "잡 목록을 상태별로 보고,	docs/spec.md 리할 수 있게, 타입, 데이터, 상태 전이, 권한, 디자인 브리프
/se:page jobs-list	디자인 플랜 → ListDetailPage 변형 + 시그니처(상태 스트립) + 3상태 + MSW
/se:api openapi.yaml	타입 클라이언트 · Query 혹 · 목 데이터 교체
/se:review	스크린샷 → 비평 → 수정 → 재확인 → design-review.md
/se:deploy	Dockerfile · CI · 환경 변수 체크리스트

웹 경험이 적은 구성원은 **요구사항 문장 → 아이덴티티 3안 중 선택 → spec 검토**, 세 번의 판단만 하면 된다.
Figma 없이 spec + 아이덴티티 + 패턴이 디자인 단계를 대체한다.

시나리오 B – 기존 프로젝트 도입 /se:adopt

Strangler 방식. 각 단계가 하나의 PR이고, 어느 단계에서 멈춰도 서비스는 동작한다. 기존 서비스가 이미 가진 개성(현재 색·톤)은 **보존**하고 형제와 충돌할 때만 조정한다.

0 감사 스택 · Node·패키지 매니저 버전 · UI 라이브러리 · 색/컴포넌트 인벤토리 · 페이지 목록 · 스크린샷 docs/adopt-report.md	1 아이덴티티 현재 개성을 보존하며 <code>/se:identity</code> 레지스트리 충돌 시만 조정 se.identity.json	2 기반 Node < 20이면 먼저 업그레이드 PR (CI 이미지 지 ·Dockerfile· <code>.nvmrc</code>). 그다음 <code>@se/tokens</code> 설치, <code>@layer se</code> 격리, ThemeProvider, 폰트. 시각 변화 0 (PR · Node) + PR · 기반	3 셸 AppShell로 네비·헤더 교체 + 시그니처 적용. 첫 눈에 보이는 일관성과 개성 PR · 셸	4 페이지 페이지 단위 codemod → Claude 보정 → 전/후 스크린샷 비평. 트래픽 많은 순 PR · 페이지 별	5 강제 eslint warn → error, 혹은 ON, 잔여 라이브러리 제거, Node 22 통일 + <code>engine-strict</code> PR · 완료
---	--	---	--	--	--

가드레일 · 거버넌스

- **기계적 강제** ESLint + PostToolUse 혹은 **시각적 강제** `/se:review` 루브릭 점수 하한(예: 80/100) 미만이면 PR 체크 실패(선택).
- **에이전트 역할 분리** critic·reviewer는 읽기 전용, 수정은 메인 세션.
- **아이덴티티 레지스트리** 서비스별 hue·시그니처·마크 등록. 신규 서비스는 충돌 검사(형제 30° · 의미 색 18°) 통과 필수. 형제와 너무 닮으면 `/se:identity` 가 거부.
- **승격 규칙** 로컬 컴포넌트/시그니처가 2개 서비스에서 반복되면 `@se/ui` 로 승격(TSDoc·story·example 필수).
- **버전** changesets + semver. 플러그인은 `@se/ui` 와 major.minor 동기.
- **문서** Storybook(사람용)은 스킴 레퍼런스와 같은 소스. 아이덴티티 갤러리로 모든 서비스 시트를 나란히 놓아 "가족 초상화"를 유지.

Node 버전 정책

기존 서비스마다 Node 버전이 제각각인 것은 "내 컴퓨터에서는 되는데"의 주범이다. 툴킷은 버전을 **고정**하고, 도입은 **하한선**으로 관리한다.

대상	규칙	강제 수단
툴킷 저장소 · 플러그인 · 레퍼런스 앱	Node 22 LTS 고정 , pnpm 9	<code>.nvmrc</code> , <code>engines.node: "22.x"</code> , <code>.npmrc engine-strict=true</code> , <code>packageManager (corepack)</code>
신규 서비스 (<code>/se:new</code>)	툴킷과 동일	템플릿에 위 파일 동봉
기존 서비스 (<code>/se:adopt</code>)	도입 중 하한 20.19 (Vite 7 · Tailwind v4 최소), 완료 시 22로 통일	0단계 감사 항목. 20 미만이면 기반 단계 전에 업그레이드 PR
모든 세션	요구 버전 미달이면 시작 시 경고	SessionStart 혹은 <code>node -v</code> 검사 → <code>.nvmrc / npm use</code> 안내

- 왜 22인가 툴체인(Vite 7, Tailwind v4, MSW 2)이 20.19+를 요구하고, 22는 2027-04까지 LTS. 20은 2026-04 EOL이라 신규 기준이 될 수 없다.
- 버전 올림은 툴킷에서 한 번에 `.nvmrc` · `engines` 변경 → 플러그인 major 업 → `SessionStart` 혹은 각 서비스에 알린다.
- 예외 레거시 런타임을 못 올리는 서비스는 도입 대상에서 제외하고 감사 리포트에 사유를 남긴다.

저장소 구조

```
se-web-toolkit/  
├─ packages/      tokens · ui · charts · eslint-plugin · codemods  
├─ templates/    app-vite-react/  
├─ examples/     reference-app · dataset-explorer · patterns/*  
├─ identities/   registry.json · <service>.identity.json  
├─ plugin/       Claude Code 플러그인  
├─ apps/         storybook/ (+ 아이덴티티 갤러리)  
├─ scripts/      gen-skill-docs.ts · check-identity.mjs  
├─ .claude-plugin/marketplace.json  
└─ docs/DESIGN.md
```

설치: `/plugin marketplace add <org>/se-web-toolkit` → `/plugin install se@se-web-toolkit`

로드맵

단계	범위	완료 기준
P1 기반 + 품질 완료	tokens(4계층, createTheme) · 컴포넌트 50개(98 export) · 시그니처 3종 · 레퍼런스 앱 3개(모니터링·조화·업무 처리) · 템플릿 + CLI · se-ui · se-design 스킴 · 명령 9 · 혹 2 · eslint-plugin · charts	세 시나리오 원본 확보. 디자인 리뷰 56 → 70/80
P2 검증 + 도입	새 세션에서 /se:new 로 4번째 서비스를 스킴만으로 생성해 품질 검증 · /se:review 루프 실사용 · /se:adopt 로 기존 서비스 1개 · codemods · 나머지 시그니처 2종	기존 서비스 1개 5단계 완료. 스킴이 만든 화면이 70/80 이상
P3 배포	@se/* npm 배포(→ 모노레포 밖 npx create-se-app) · Storybook·아이덴티티 갤러리 · 기여 가이드 · 플러그인 버전 정책	웹 비전공 구성원이 혼자 서비스 1개 출시

셋업 → 첫 화면 배포 시간

/se:audit 준수율 · 디자인 점수

서비스 간 hue·시그니처 중복 0

신규 구성원 첫 PR까지 시간

핵심 결정과 이유

결정	대안	선택 이유
아이덴티티 2층 (코어 고정 + 서비스 슬롯 필수)	단일 테마 / 완전 자유	단일 테마는 양산형, 완전 자유는 일관성 붕괴. 슬롯 모델만이 둘을 동시에 만족한다
시그니처를 검증된 목록에서 선택	매번 자유 발명	발명은 품질 편차가 크다. 목록은 승격 규칙으로 성장한다
디자인 플랜 + 스크린샷 비평 루프	코드 규칙만	코드 규칙은 "틀리지 않음"을 보장할 뿐 "잘 만듦"을 보장하지 않는다
레퍼런스 앱에 디자인 공수 집중	컴포넌트만 잘 만들기	Claude는 예제를 변형한다. 예제 품질 = 결과 품질
npm 패키지 배포	shadcn식 소스 복사	문제의 본질이 "갈라짐". 개성은 토큰 슬롯으로 내고, 코드 분기로 내지 않는다
shadcn/Radix API 관례	독자 API	Claude가 이미 아는 이름 → 생성 정확도, 낮은 학습 비용
Tailwind v4 + CSS 변수 토큰	CSS Modules / CSS-in-JS	생성 품질 최상. 토큰이 CSS 변수라 기존 프로젝트에도 주입 가능
문서를 코드에서 생성	손으로 쓴 스킴 문서	스킬 드리프트가 가장 큰 실패 요인. 생성 + CI로 차단
규칙을 린터로	긴 가이드 문서	LLM은 문서를 잊지만 exit code는 못 잊는다
백엔드 무관 (OpenAPI + MSW)	백엔드 표준화	백엔드는 비표준이 현실. UI 먼저 완성 후 연결하는 흐름이 소수 인원에 유리
Node 22 LTS 고정 + engine-strict	범위 허용(≥ 18) / 프로젝트별 자율	툴체인 하한이 20.19이고 버전 혼재가 재현 불가 버그의 주범. 고정 + 혹 검사가 가장 싸다
액센트 hue를 의미 색과 18° 이상 떨어뜨림	hue 완전 자유	가까우면 액센트 칩이 경고·오류로 읽힌다. 아이덴티티가 의미 체계를 흔들면 안 된다

리스크

- 시그니처·hue 슬롯 고갈 → 새 시그니처 승격, hue 밴드 세분화로 슬롯을 늘린다.
- 비평 루프 비용(시간·토큰) → `/se:page` 는 스크린샷 1회만, 전체 루프는 `/se:review` 에서만.
- 레퍼런스 앱 품질 확보 → P1에 디자이너 1회 참여, 또는 팀 내 디자인 감각 있는 인원이 최종 승인.
- 기존 서비스 UI 라이브러리 혼재(MUI/antd) → codemod 80% + `se-migrator` .
- 기존 서비스 Node 버전 상이(18 이하) → 0단계 감사에서 드러나고, 기반 단계 전 별도 업그레이드 PR로 분리. 못 올리면 도입 제외.